

Davranış Uzayı Analizinden Operasyonel Düzeltmeye: Yedi Adımlı Yöntem

Ömer Faruk Yavuz

May 2026

Davranış Uzayı Analizinden Operasyonel Düzeltmeye: Yedi Adımlı Yöntem

Bu çalışma, soyut bir sistem analizinden başlayıp operasyonel bir kod düzeltmesine ulaşan, örnekten bağımsız bir yöntem önermektedir. Söz konusu yöntem, daha önce ele alınan davranış uzayı kavramının pratik bir uygulamasını ifade eder. Yöntem yedi adımdan oluşur ve her adım, bir öncekinden daha somut bir çıktı üretir.

Adımlar

Sistemi Haritalandır

İlk adımda sistemin yapısal profili çıkarılır: girdi eksenleri, akış dalları, durum makinesi, kalıcılık (persistans) katmanı ve dış kenarlar tanımlanır. Bu aşamada henüz hata tespiti yapılmaz; yalnızca sistemin geometrik şekli kavranır. Adımın çıktısı, bir paragraflık bir sistem profilidir.

İnvariantları Listele

İkinci adımda, sistemin hiçbir durumda ihlal etmemesi gereken kurallar açıkça yazılır ve önem sırasına dizilir. İnvariantlar, sonraki adımlarda kurulacak test stratejisinin omurgasını oluşturur. Adımın çıktısı, numaralandırılmış bir invariant listesidir.

İhlal Yollarını Çıkar

Üçüncü adımda her invariantın altına inilir ve hangi olay sıralamalarının ilgili invariantı ihlal edebileceği araştırılır. Bu adımda varsayım yerine kodun doğrudan okunması esastır; gerçek satır numaraları ve dosya yolları kaydedilir. Her ihlal yolunun karşısına, “bir savunma mekanizması var mı, varsa hangisi?” notu düşülür. Adımın çıktısı, ihlal yolunu savunma mekanizmasıyla eşleyen bir matristir.

İhlal Yollarını Test Türüne Eşle

Dördüncü adımda her ihlal yolu, kendisine uygun test türüyle eşleştirilir. Bazı ihlaller klasik birim testle yakalanabilirken, bazıları özellik tabanlı test (property-based testing), durum kapsamı (state coverage) veya kaos enjeksiyonu (chaos injection) gerektirir. Test türü, ihlalin şekline göre seçilir; zira tek bir paradigma tüm ihlalleri kapsayamaz. Adımın çıktısı, ihlal ile test türü arasında kurulan eşleme tablosudur.

Önceliklendirme Tablosu Kur

Beşinci adımda her ihlal için dört değer hesaplanır: gerçekleşme olasılığı, canlı sistemde tespit zorluğu, düzeltme maliyeti ve test maliyeti. Bu değerler birlikte değerlendirilerek bir öncelik sıralaması üretilir. Adımın çıktısı, risk-efor matrisine göre sıralanmış bir listedir.

Sentezi Gözden Geçir

Altıncı adımda, üretilen test stratejisi eleştirel biçimde sorgulanır: strateji gerçekten yeterli midir? Bir test, yakalaması gereken davranış uzayını yeterince geniş kapsıyor mudur, yoksa fazla dar mı tanımlanmıştır? Önceliklendirme doğru mudur, yoksa “küçük hata” olarak etiketlenen bir vaka, aslında daha geniş bir yapısal kök nedenin yalnızca bir yüzü müdür? Bu adım, analiz hatalarının tespit edildiği düzeydir. Adımın çıktısı, tashih edilmiş analizdir.

En Kritik Tek Vakayı Uygula

Yedinci adımda şu soru sorulur: “Şu anda canlı sistemde gerçekleşebilecek en kötü ihlal hangisidir?” Belirlenen kök neden düzeltilir, bir regresyon testi eklenir ve dokümantasyon güncellenir. Listenin tamamı tek seferde uygulanmaz; geri kalan vakalar sonraki sprintlere dağıtılır. Adımın çıktısı; çalışan kod, regresyon testi ve güncellenmiş notlardır.

Yöntemin Omurgası: Üç İlke

Yedi adımlık iskelet, birbirini destekleyen üç ilkeye dayanır.

Soyuttan Somuta Zorlanmış Geçiş

Her adım, bir öncekinden daha somut olmak zorundadır; kavramsal düzeyde kalmak bu yöntemde kabul edilmez. Her aşamada satır numarası, test senaryosu ya da dosya yolu gibi izlenebilir bir çıktı üretilmelidir. Aksi hâlde adımlar arası bağ kopar ve yöntem teorik bir egzersize dönüşür.

Geriye Dönük Tashih Zorunluluğu

Altıncı adım atlanmamalıdır. Pratikte mühendisler çoğunlukla dördüncü ve beşinci adımda durur ve ürettikleri analizi yeterli sayar. Oysa asıl içgörüler altıncı adımda ortaya çıkar; “küçük olarak etiketlenen bir vakanın, yapısal bir kök nedenin tek yüzü olduğu” türünden tashihler bu düzeyde yapılır. Bu adım, yöntemin diğer test stratejisi yaklaşımlarından ayrıldığı kritik nokta olarak değerlendirilebilir.

Tek Nokta Uygulaması

Yedinci adımda tüm listenin bir defada uygulanması hedeflenmez. Bunun yerine “bugün canlıda gerçekleşebilecek en kritik tek vaka” belirlenir ve önce o çözülür; geri kalan vakalar sprintlere dağıtılır. Aksi hâlde liste sonsuza kadar liste olarak kalır ve hiçbir gerçek düzeltme üretilmez. Bu ilke, analizin yalnızca analiz olarak kalmamasını, kod tabanına filen dokunmasını güvence altına alır.

Uygulanabilirlik

Yöntemin Uygun Olduğu Sistemler

Söz konusu yöntem, durum uzayı matematiksel olarak temsil edilebilen kod tabanlarında işler. Örnekler arasında WebSocket protokol uygulamaları, dağıtık sistemler, durum güdümlü kullanıcı arayüzleri, asenkron orkestrasyon mekanizmaları ve kalıcılık katmanlı uygulamalar yer alır. Modern yazılımın büyük çoğunluğu bu kategoriye girer.

Yöntemin Aşırı Yatırım Sayıldığı Sistemler

Tümüyle kategorik iş kurallarından oluşan yapılar (örneğin form doğrulama veya basit CRUD uçları) ile saf işlevsel dönüşümler (kütüphane fonksiyonları), bu yöntemin maliyetini hak etmez. Bu tür kod parçaları, klasik birim testler ya da özellik tabanlı test ile yeterince kapsanır; davranış uzayı analizi ise söz konusu durumlarda gereksiz bir yük yaratır. Yöntem, davranış uzayının karmaşıklığıyla orantılı bir araçtır.

Yöntemin Tanımlayıcı Özelliği

Söz konusu yöntem, geleneksel “hata bul ve düzelt” döngüsünden farklı bir paradigma önerir. Burada izlenen sıra şudur: önce sistemin nereden bozulabileceğinin haritası çıkarılır, ardından bu olası bozulmalar önceliklendirilir, sonra haritanın doğruluğu sorgulanır ve nihayet en kritik noktaya odaklanılır. Bu yaklaşımda hatalar, sürecin girdisi değil çıktısı olarak ortaya çıkar. Yöntem bu yönüyle, reaktif hata yönetiminden proaktif sistem analizine geçişi temsil eder.